

WEEK 3-Spring REST using Spring Boot 3

Name : Guneet Kaur Juneja

Superset ID: 8003337

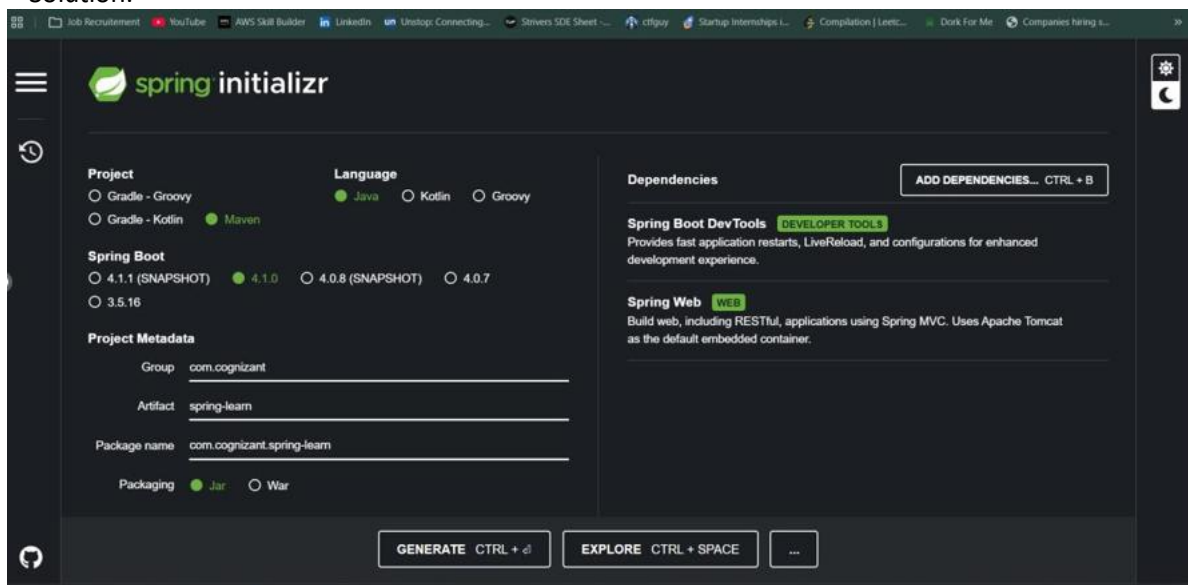
Hands on 1 : Create a Spring Web Project using Maven

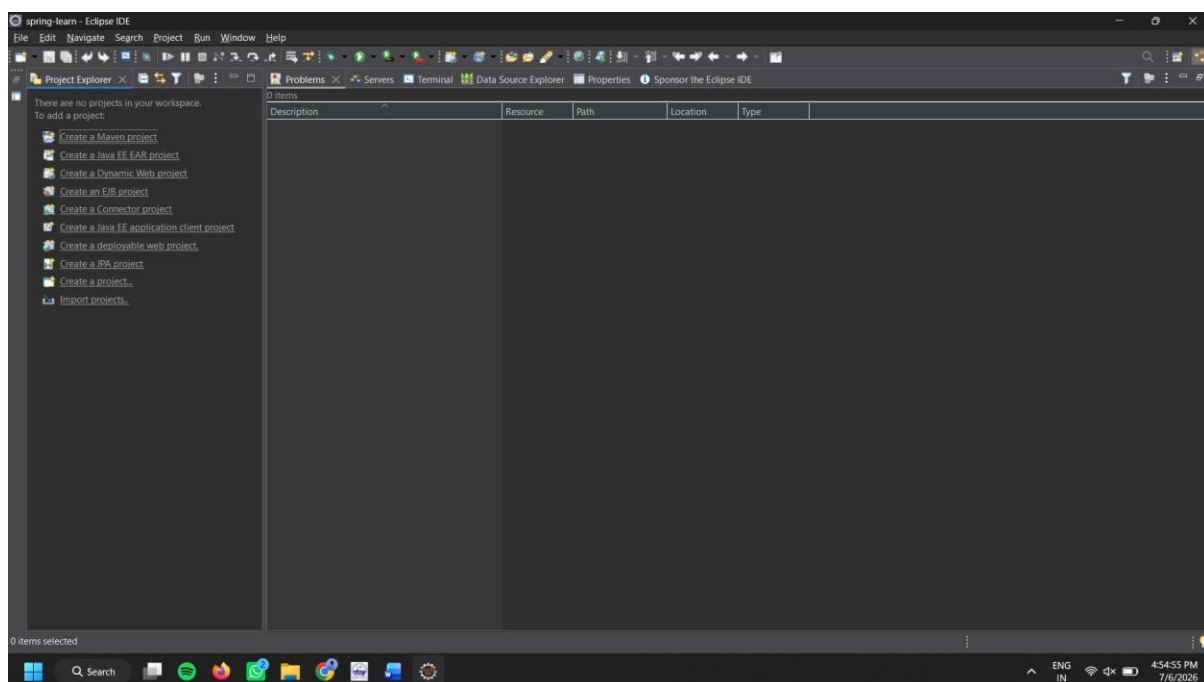
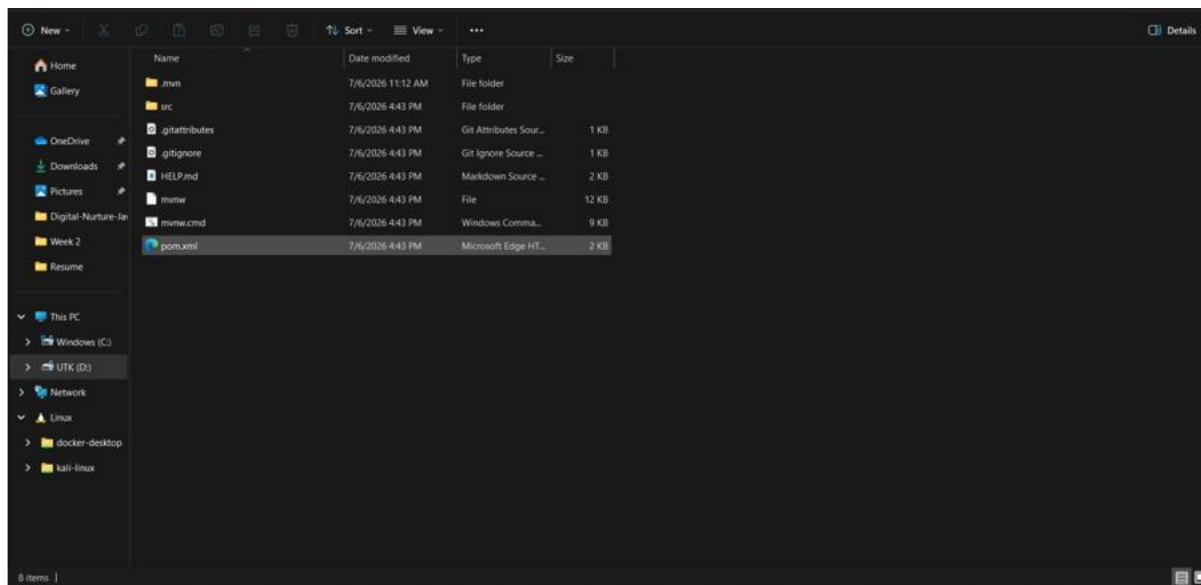
Follow steps below to create a project:

1. Go to <https://start.spring.io/>
2. Change Group as "com.cognizant"
3. Change Artifact Id as "spring-learn"
4. Select Spring Boot DevTools and Spring Web
5. Create and download the project as zip
6. Extract the zip in root folder to Eclipse Workspace
7. Build the project using 'mvn clean package -Dhttp.proxyHost=proxy.cognizant.com -Dhttp.proxyPort=6050 -Dhttps.proxyHost=proxy.cognizant.com -Dhttps.proxyPort=6050 -Dhttp.proxyUser=123456' command in command line
8. Import the project in Eclipse "File > Import > Maven > Existing Maven Projects > Click Browse and select extracted folder > Finish"
9. Include logs to verify if main() method of SpringLearnApplication.

Run the SpringLearnApplication class.

Solution:





Hands on 2 : Spring Core – Load Country from Spring Configuration XML

Spring Core – Load SimpleDateFormat from Spring Configuration XML

SimpleDateFormat with the pattern 'dd/MM/yyyy' is created in multiple places of an application. To avoid creation of SimpleDateFormat in multiple places, define a bean in Spring XML Configuration file and retrieve the date.

Follow steps below to implement::

- Create spring configuration file date-format.xml in src/main/resources folder of 'spring-learn' project
- Open <https://docs.spring.io/spring-framework/docs/current/spring-framework-reference/core.html#beans-factory-metadata>
- Copy the XML defined in the section of previous step URL and paste it into date-format.xml
- Define bean tag in the XML with for date format. Refer code below.

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="dateFormat" class="java.text.SimpleDateFormat">
        <constructor-arg value="dd/MM/yyyy" />
    </bean>

</beans>
```

- Create new method displayDate() in SpringLearnApplication.java
- In displayDate() method create the ApplicationContext. Refer code below::

```
ApplicationContext context = new ClassPathXmlApplicationContext("date-format.xml");
```

- Get the dateFormat using getBean() method. Refer code below.

```
SimpleDateFormat format = context.getBean("dateFormat", SimpleDateFormat.class);
```

- Using the format variable try to parse string '31/12/2018' to Date class and display the result using System.out.println.
- Run the application as 'Java Application' and check the result in console log output.

SpringBootApplication.java:

```
package com.cognizant.spring_learn;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;

@SpringBootApplication
public class SpringLearnApplication {

    public void displayDate() throws ParseException {
        ClassPathXmlApplicationContext context = new ClassPathXmlApplicationContext("date-
format.xml");
        SimpleDateFormat format = context.getBean("dateFormat", SimpleDateFormat.class);
        Date date = format.parse("31/12/2018");
        System.out.println(date);
        context.close();
    }

    public static void main(String[] args) throws ParseException {
        SpringApplication.run(SpringLearnApplication.class, args);
        SpringLearnApplication app = new SpringLearnApplication();
        app.displayDate();
    }
}
```

Hands on 3 : Hello World RESTful Web Service

Write a REST service in the spring learn application created earlier, that returns the text "Hello World!!" using Spring Web Framework. Refer details below:

Method: GET

URL: /hello

Controller: com.cognizant.spring-learn.controller.HelloController

Method Signature: public String sayHello()

Method Implementation: return hard coded string "Hello World!!"

Sample Request: http://localhost:8083/hello

Sample Response: Hello World!!

IMPORTANT NOTE: Don't forget to include start and end log in the sayHello() method.

Try the URL http://localhost:8083/hello in both chrome browser and postman.

```
package com.cognizant.spring_learn.controller;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class HelloController {
```

```
    private static final Logger LOGGER =
```

```
        LoggerFactory.getLogger(HelloController.class);
```

```
    @GetMapping("/hello")
```

```
    public String sayHello()
```

```
    {
```

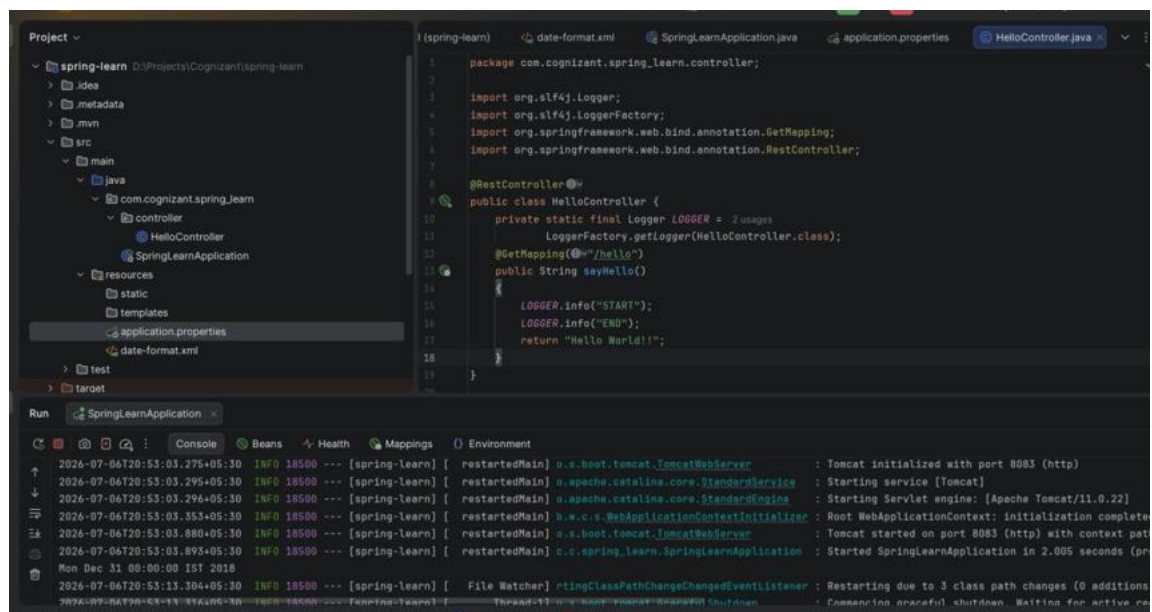
```
        LOGGER.info("START");
```

```
        LOGGER.info("END");
```

```
        return "Hello World!!!";
```

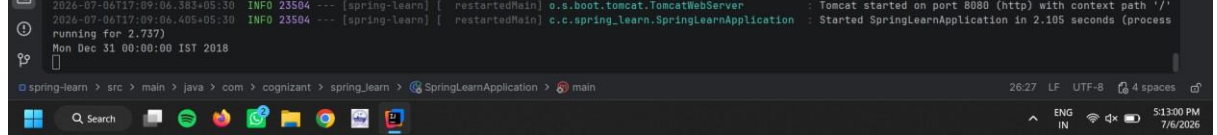
```
    }
```

```
}
```





Desktop 1



REST - Country Web Service

Write a REST service that returns India country details in the earlier created spring learn application.

URL: /country

Controller: com.cognizant.spring-learn.controller.CountryController

Method Annotation: @RequestMapping

Method Name: getCountryIndia()

Method Implementation: Load India bean from spring xml configuration and return

Sample Request: <http://localhost:8083/country>

Sample Response:

```
{  
  "code": "IN",  
  "name": "India"  
}
```

Solution:

Country.java

```
package com.cognizant.spring_learn.model;
```

```
public class Country {  
  
    private String code;  
    private String name;  
  
    public Country() {  
    }  
  
    public Country(String code, String name) {  
        this.code = code;  
        this.name = name;  
    }  
  
    public String getCode() {  
        return code;  
    }  
  
    public void setCode(String code) {  
        this.code = code;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```


CountryController.java

```
package com.cognizant.spring_learn.controller;
```

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
import com.cognizant.spring_learn.model.Country;
```

```
@RestController
```

```
public class CountryController {
```

```
    private static final Logger LOGGER = LoggerFactory.getLogger(CountryController.class);
```

```
    @RequestMapping("/country")
```

```
    public Country getCountryIndia() {
```

```
        LOGGER.info("START");
```

```
        ClassPathXmlApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
```

```
        Country country = context.getBean("country", Country.class);
```

```
        LOGGER.info("END");
```

```
        context.close();
```

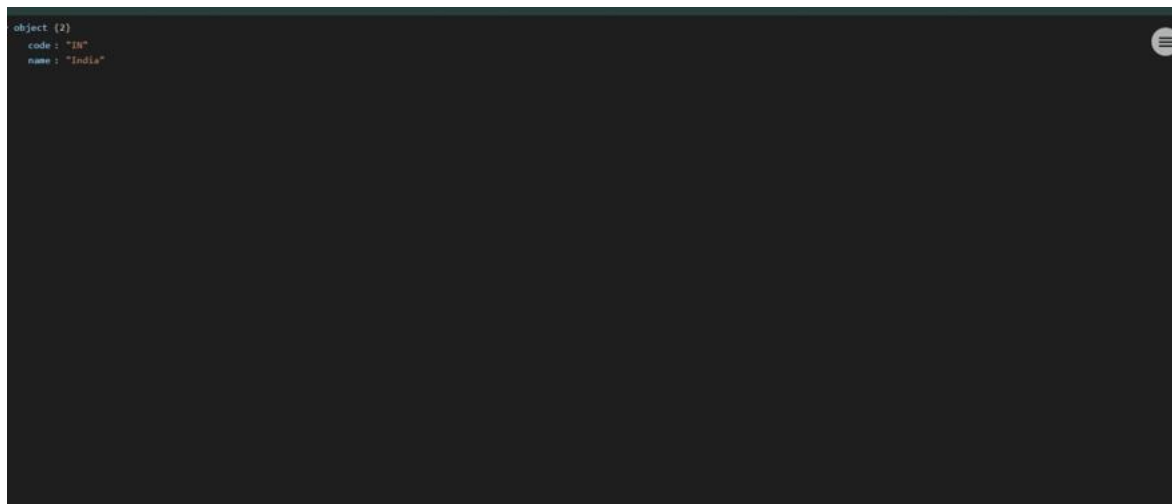
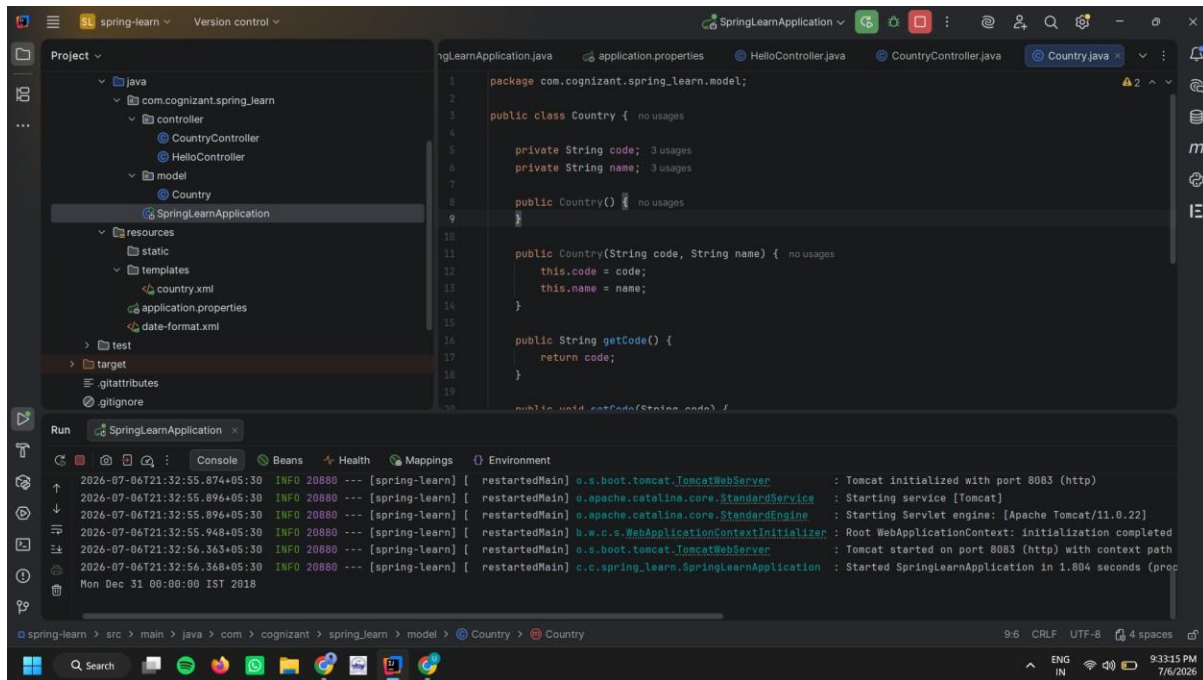
```
        return country;
```

```
    }
```

```
}
```

The screenshot displays an IDE interface with the following components:

- Project Explorer (Left):** Shows the project structure for 'spring-learn'. The 'resources' folder is expanded, revealing 'country.xml' and 'application.properties'.
- Editor (Center):** Displays the code for 'CountryController.java'. The code includes imports for logging and Spring context, a REST controller annotation, and a method 'getCountryIndia()' that initializes a 'ClassPathXmlApplicationContext' to load 'country.xml' and retrieve a 'Country' bean.
- Console (Bottom):** Shows the application's startup logs. Key messages include: 'Tomcat initialized with port 8083 (http)', 'Starting service [Tomcat]', 'Starting Servlet engine: [Apache Tomcat/11.0.22]', 'Root WebApplicationContext: initialization completed', and 'Tomcat started on port 8083 (http) with context path'. The application is identified as 'c.c.spring_learn.SpringLearnApplication'.



CountryController.java

```
package com.cognizant.spring_learn.controller;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.cognizant.spring_learn.model.Country;

@RestController
public class CountryController {

    private static final Logger LOGGER = LoggerFactory.getLogger(CountryController.class);
    @RequestMapping("/country")
    public Country getCountryIndia() {

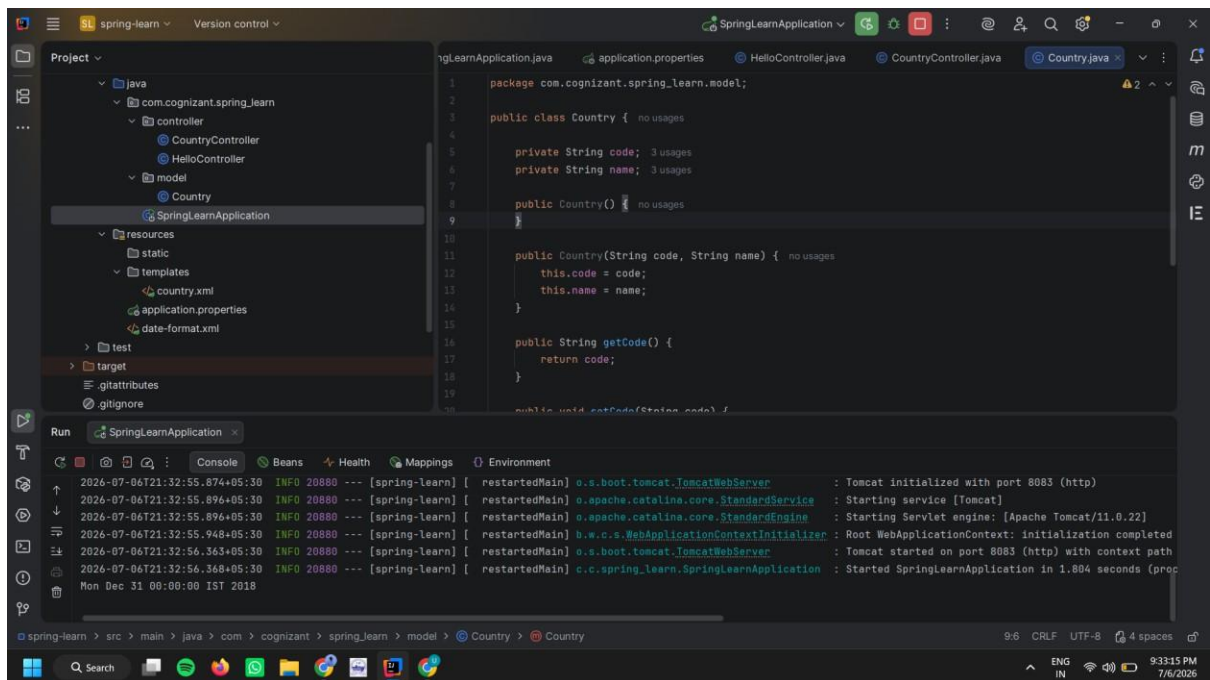
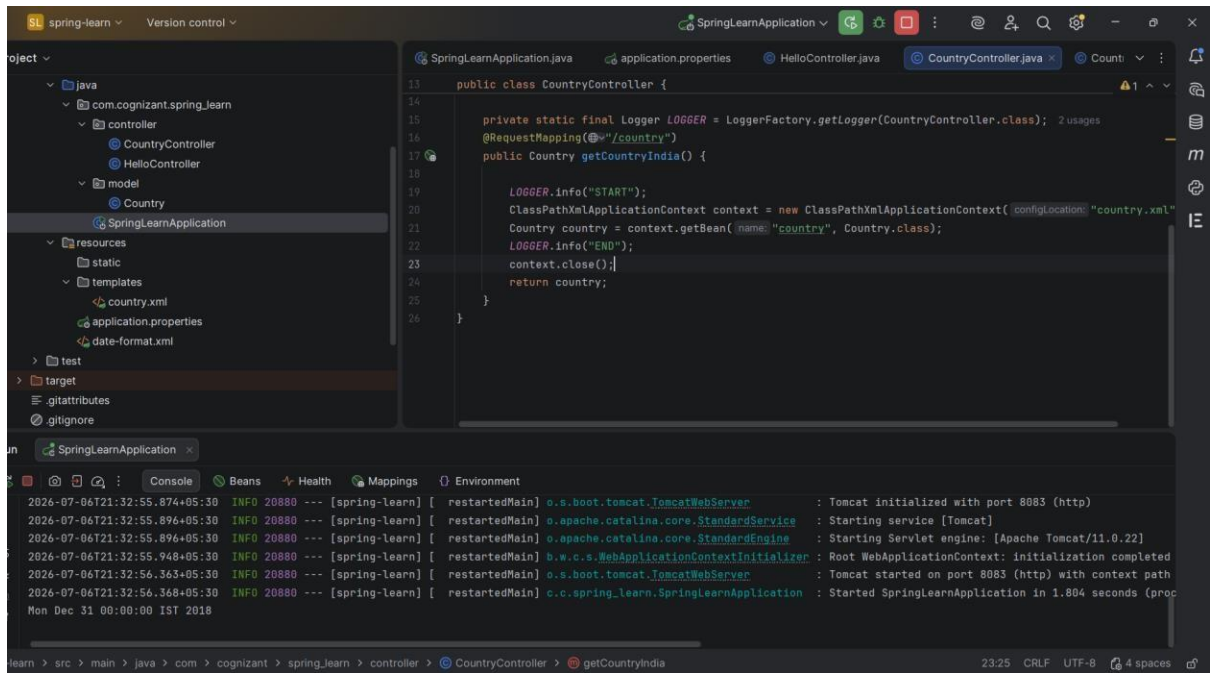
        LOGGER.info("START");
        ClassPathXmlApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("country", Country.class);
        LOGGER.info("END");
        context.close();
        return country;
    }

    @RequestMapping("/countries")
    public Country getAllCountries() {

        LOGGER.info("START");
        ClassPathXmlApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("country", Country.class);
        LOGGER.info("END");
        context.close();
        return country;
    }

}

```



```
object (2)
  code : "IN"
  name : "India"
```

Create authentication service that returns JWT

As part of first step of JWT process, the user credentials needs to be sent to authentication service request that generates and returns the JWT.

Ideally when the below curl command is executed that calls the new authentication service, the token should be responded. Kindly note that the credentials are passed using -u option.

Request

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

Response

```
{"token":"eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJ1c2VyliwiaWF0IjoxNTcwMzc5NDc0LCJleHAiOiE1NzAzODAzODR9.t3LRvICV-hwKfoqZYlaVQqEUiBloWcWn0ft3tgv0dL0"}
```

This can be incorporated as three major steps:

- Create authentication controller and configure it in SecurityConfig
- Read Authorization header and decode the username and password
- Generate token based on the user retrieved in the previous step

Solution :

AuthenticateController.java :

```
package com.cognizant.spring_learn.controller;

import java.util.HashMap;
import java.util.Map;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AuthenticationController {

    private static final Logger LOGGER = LoggerFactory.getLogger(AuthenticationController.class);

    @GetMapping("/authenticate")
    public Map<String, String> authenticate(@RequestHeader("Authorization") String authHeader) {

        LOGGER.info("START");

        LOGGER.debug("Authorization Header: {}", authHeader);

        Map<String, String> map = new HashMap<>();
        map.put("token", "");

        LOGGER.info("END");

        return map;
    }
}
```

SecurityConfig.java

```
package com.cognizant.spring_learn.controller;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.SecurityFilterChain;
```

```

@Configuration
public class SecurityConfig {

    @Bean
    public UserDetailsService userDetailsService() {

        UserDetails user = User.withUsername("user")
            .password("{noop}pwd")
            .roles("USER")
            .build();

        UserDetails admin = User.withUsername("admin")
            .password("{noop}pwd")
            .roles("ADMIN")
            .build();

        return new InMemoryUserDetailsManager(user, admin);
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http
            .csrf(csrf -> csrf.disable())

            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/countries").hasRole("USER")
                .requestMatchers("/authenticate").hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated()
            )

            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}

```

The screenshot shows an IDE with a project named 'spring-learn'. The project structure is as follows:

- src
 - main
 - java
 - com.cognizant.spring.learn
 - controller
 - AuthenticationController
 - CountryController
 - HelloController
 - SecurityConfig
 - model
 - Country
 - resources
 - static
 - templates
 - application.properties
 - country.xml
 - date-format.xml

The 'SecurityConfig.java' file is open, showing the following code:

```
1 package com.cognizant.spring.learn.controller;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.config.Customizer;
6 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
7 import org.springframework.security.core.userdetails.User;
8 import org.springframework.security.core.userdetails.UserDetails;
9 import org.springframework.security.provisioning.InMemoryUserDetailsManager;
10 import org.springframework.security.core.userdetails.UserDetailsService;
11 import org.springframework.security.web.SecurityFilterChain;
12
13 @Configuration
14 public class SecurityConfig {
15
16     @Bean
17     public UserDetailsService userDetailsService() {
18         UserDetails user = User.withUsername("user")
19             .password("{noop}password").build();
20     }
21 }
```

The terminal shows the following output:

```
unning for 1.99s
Mon Dec 31 00:00:00 IST 2018
2026-07-06T22:02:39.584+05:30 INFO 18192 --- [spring-learn] [nio-8083-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-07-06T22:02:39.587+05:30 INFO 18192 --- [spring-learn] [nio-8083-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-07-06T22:02:39.591+05:30 INFO 18192 --- [spring-learn] [nio-8083-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 3 ms
2026-07-06T22:02:49.493+05:30 INFO 18192 --- [spring-learn] [nio-8083-exec-7] c.c.s.c.AuthenticationController : START
2026-07-06T22:02:49.494+05:30 INFO 18192 --- [spring-learn] [nio-8083-exec-7] c.c.s.c.AuthenticationController : END
2026-07-06T22:02:57.392+05:30 INFO 18192 --- [spring-learn] [nio-8083-exec-2] c.c.s.c.AuthenticationController : START
2026-07-06T22:02:57.393+05:30 INFO 18192 --- [spring-learn] [nio-8083-exec-2] c.c.s.c.AuthenticationController : END
2026-07-06T22:03:04.365+05:30 INFO 18192 --- [spring-learn] [nio-8083-exec-4] c.c.s.c.AuthenticationController : START
2026-07-06T22:03:04.366+05:30 INFO 18192 --- [spring-learn] [nio-8083-exec-4] c.c.s.c.AuthenticationController : END
```

The screenshot shows a terminal window with the following output:

```
object (1)
  token: ""
```